

EXPT : 4	EXPLORATORY DATA ANALYSIS (EDA) – DATA INSPECTION, FILTERING, STATISTICAL ANALYSIS AND CORRELATION
DATE :	

AIM

To perform data inspection, filtering, grouping, aggregation, descriptive statistical analysis, and correlation analysis using Pandas and visualize the results using Matplotlib.

ALGORITHM 1

Dataset Inspection

1. Import dataset
2. Display dataset
3. Check dimensions
4. Display data types
5. Generate summary

PROGRAM 1

```
import pandas as pd  
df=pd.read_csv(r"C:\Users\Keertu\Downloads\tourism_dataset_5000.csv")  
print(df)
```

OUTPUT

	Sites Visited	Tour Duration	\
0	['Eiffel Tower', 'Great Wall of China', 'Taj M...]	7	
1	['Great Wall of China']	1	
2	['Eiffel Tower']	2	
3	['Machu Picchu', 'Taj Mahal']	5	
4	['Machu Picchu', 'Taj Mahal', 'Great Wall of C...]	7	
...	...	...	
4995	['Great Wall of China']	2	
4996	['Colosseum']	1	
4997	['Taj Mahal']	1	
4998	['Taj Mahal', 'Great Wall of China', 'Colosseum']	7	
4999	['Great Wall of China']	1	

	Route ID	Tourist Rating	System Response Time	Recommendation Accuracy	\
0	1000	1.6	3.73	97	
1	2000	2.6	2.89	90	
2	3000	1.7	2.22	94	
3	4000	2.0	2.34	92	
4	5000	3.7	2.00	96	
...	...	...	...	...	
4995	4996000	3.9	2.53	100	
4996	4997000	4.3	1.53	100	
4997	4998000	3.2	3.99	89	
4998	4999000	2.5	2.13	98	
4999	5000000	1.9	2.99	92	

	VR Experience Quality	Satisfaction
0	4.5	3
1	4.5	3
2	4.7	3
3	4.7	3
4	4.8	4
...	...	...
4995	4.4	4
4996	5.0	4
4997	4.4	3
4998	4.4	3
4999	4.8	3

[5000 rows x 14 columns]

PROGRAM 2

```
print(df.shape)
```

OUTPUT

```
(5000, 14)
```

PROGRAM 3:

```
df.dtypes
```

OUTPUT

Tourist ID	int64
Age	int64
Interests	object
Preferred Tour Duration	int64
Accessibility	bool
Site Name	object
Sites Visited	object
Tour Duration	int64
Route ID	int64
Tourist Rating	float64
System Response Time	float64
Recommendation Accuracy	int64
VR Experience Quality	float64
Satisfaction	int64
dtype:	object

ALGORITHM 2

Filtering Data

1. Specify condition
2. Apply filter
3. Display records

PROGRAM 4 :

```
high_rating = df[df["Tourist Rating"] > 4]
```

```
print(high_rating)
```

OUTPUT

	Preferred Tour Duration	Accessibility	Site Name \
6	7	False	Eiffel Tower
13	8	True	Colosseum
22	8	True	Great Wall of China
28	7	True	Great Wall of China
29	6	True	Eiffel Tower
...	...	...	...
4981	5	True	Great Wall of China
4983	3	False	Taj Mahal
4991	6	True	Eiffel Tower
4994	4	False	Eiffel Tower
4996	7	True	Great Wall of China

	Sites Visited	Tour Duration \
6	['Eiffel Tower', 'Colosseum', 'Taj Mahal']	6
13	['Eiffel Tower']	1
22	['Colosseum']	3
28	['Taj Mahal', 'Great Wall of China', 'Eiffel T...']	8
29	['Taj Mahal', 'Great Wall of China']	2
...	...	...
4981	['Machu Picchu', 'Eiffel Tower', 'Taj Mahal']	4
4983	['Colosseum', 'Taj Mahal']	2
4991	['Machu Picchu', 'Taj Mahal']	4
4994	['Taj Mahal', 'Machu Picchu']	5
4996	['Colosseum']	1

	Route ID	Tourist Rating	System Response Time	Recommendation Accuracy \
6	7000	4.1	4.66	93
13	14000	4.3	3.48	91
22	23000	4.2	1.97	88
28	29000	4.1	4.57	91
29	30000	4.4	2.18	95
...	...	...	...	...
4981	4982000	4.6	3.64	90
4983	4984000	4.8	3.43	87
4991	4992000	5.0	2.25	90
4994	4995000	4.3	2.98	97
4996	4997000	4.3	1.53	100

	VR Experience Quality	Satisfaction
6	4.5	4
13	4.4	4
22	5.0	4
28	4.6	4
29	4.9	4
...	...	...
4981	4.5	5
4983	4.7	5
4991	4.7	5
4994	4.6	4
4996	5.0	4

```
[1213 rows x 14 columns]
```

PROGRAM 5

```
history_tourists = df[df["Interests"] == "History"]
```

```
print(history_tourists)
```

OUTPUT

```
Empty DataFrame
Columns: [Tourist ID, Age, Interests, Preferred Tour Duration, Accessibility, Site Name, Sites Visited, Tour Duration, Route ID, Tourist Rating, System Response Time, Recommendation Accuracy, VR Experience Quality, Satisfaction]
Index: []
```

ALGORITHM 3

Sorting Data

1. Select column
2. Apply sort_values()
3. Display sorted result

PROGRAM 6

```
sorted_df = df.sort_values(  
    by="Satisfaction",  
    ascending=False  
)
```

```
print(sorted_df)
```

OUTPUT

Tourist ID Age			Interests \
621	622	52	['Architecture']
3245	3246	20	['Architecture', 'Art']
3897	3898	39	['History', 'Architecture']
4647	4648	48	['Art', 'Architecture']
3504	3505	51	['Nature', 'Cultural', 'Art']
...	...	...	...
2049	2050	28	['Art']
2052	2053	36	['History', 'Architecture', 'Nature']
2053	2054	65	['Art', 'Architecture', 'History']
2056	2057	38	['Architecture', 'Art']
4999	5000	32	['Nature']
Preferred Tour Duration Accessibility			Site Name \
621		3	False Eiffel Tower
3245		6	True Colosseum
3897		6	False Great Wall of China
4647		5	False Colosseum
3504		3	True Taj Mahal
...		...	...
2049		7	False Machu Picchu
2052		5	False Taj Mahal
2053		6	False Taj Mahal
2056		4	False Taj Mahal
4999		4	False Colosseum
Sites Visited			Tour Duration \
621		['Eiffel Tower', 'Colosseum']	5
3245		['Colosseum']	3
3897		['Eiffel Tower', 'Colosseum', 'Taj Mahal']	7
4647		['Colosseum', 'Great Wall of China', 'Machu Picchu']	6
3504		['Taj Mahal']	3
...		...	...
2049		['Taj Mahal', 'Eiffel Tower', 'Machu Picchu']	8
2052		['Taj Mahal']	1
2053		['Eiffel Tower']	3
2056		['Machu Picchu']	2
4999		['Great Wall of China']	1
Route ID	Tourist Rating	System Response Time	Recommendation Accuracy \
621	622000	4.6	4.90 98
3245	3246000	4.5	2.65 91
3897	3898000	4.5	4.74 93
4647	4648000	4.6	3.35 87
3504	3505000	4.5	3.90 87
...	...	...	...
2049	2050000	3.1	2.86 87
2052	2053000	2.7	4.25 90
2053	2054000	2.0	2.81 89
2056	2057000	2.9	4.13 95
4999	5000000	1.9	2.99 92
VR Experience Quality		Satisfaction	
621	4.3	5	
3245	4.7	5	
3897	4.5	5	
4647	4.3	5	
3504	4.8	5	
...	...	...	
2049	4.7	3	
2052	4.3	3	

VISUALIZATION

```
print("Mean =",  
      df["Satisfaction"].mean())  
  
print("Median =",  
      df["Satisfaction"].median())  
  
print("Mode =",  
      df["Satisfaction"].mode())  
  
print("Standard Deviation =",  
      df["Satisfaction"].std())
```

OUTPUT

```
Mean = 3.5342  
Median = 3.0  
Mode = 0    3  
Name: Satisfaction, dtype: int64  
Standard Deviation = 0.7360290128395618
```

ALGORITHM 4

Statistical Analysis

1. Select numerical column
2. Calculate Mean
3. Calculate Median
4. Calculate Mode
5. Calculate Standard Deviation

PROGRAM 7

```
print(df.describe())
```

OUTPUT

	Tourist ID	Age	Preferred Tour Duration	Tour Duration \
count	5000.000000	5000.000000	5000.000000	5000.000000
mean	2500.500000	43.886000	5.52280	3.97720
std	1443.520003	15.268858	1.71186	1.99967
min	1.000000	18.000000	3.00000	1.00000
25%	1250.750000	30.000000	4.00000	2.00000
50%	2500.500000	44.000000	6.00000	4.00000
75%	3750.250000	57.000000	7.00000	5.00000
max	5000.000000	70.000000	8.00000	9.00000

	Route ID	Tourist Rating	System Response Time \
count	5.000000e+03	5000.000000	5000.000000
mean	2.500500e+06	3.002740	3.238802
std	1.443520e+06	1.164353	1.016123
min	1.000000e+03	1.000000	1.500000
25%	1.250750e+06	2.000000	2.360000
50%	2.500500e+06	3.000000	3.230000
75%	3.750250e+06	4.000000	4.130000
max	5.000000e+06	5.000000	5.000000

	Recommendation Accuracy	VR Experience Quality	Satisfaction
count	5000.000000	5000.000000	5000.000000
mean	92.492200	4.493160	3.534200
std	4.616081	0.290596	0.736029
min	85.000000	4.000000	3.000000
25%	88.000000	4.200000	3.000000
50%	92.000000	4.500000	3.000000
75%	97.000000	4.700000	4.000000
max	100.000000	5.000000	5.000000

PROGRAM 8

```
accessibility_avg = df.groupby(
    "Accessibility"
)["Tourist Rating"].mean()
```

```
print(accessibility_avg)
```

OUTPUT

```
Accessibility
False    3.006914
True     2.998461
Name: Tourist Rating, dtype: float64
```

ALGORITHM 5

Grouping and Aggregation

1. Select Department Column
2. Apply groupby()
3. Calculate Average Marks
4. Display Result

PROGRAM 9

```
corr = df[[
    "Recommendation Accuracy",
```

"Satisfaction"

]].corr()

print(corr)

OUTPUT

	Recommendation Accuracy	Satisfaction
Recommendation Accuracy	1.000000	-0.002453
Satisfaction	-0.002453	1.000000

VISUALIZATION

import matplotlib.pyplot as plt

access_avg = df.groupby("Accessibility")["Satisfaction"].mean()

access_avg.plot(kind="bar", figsize=(8,5))

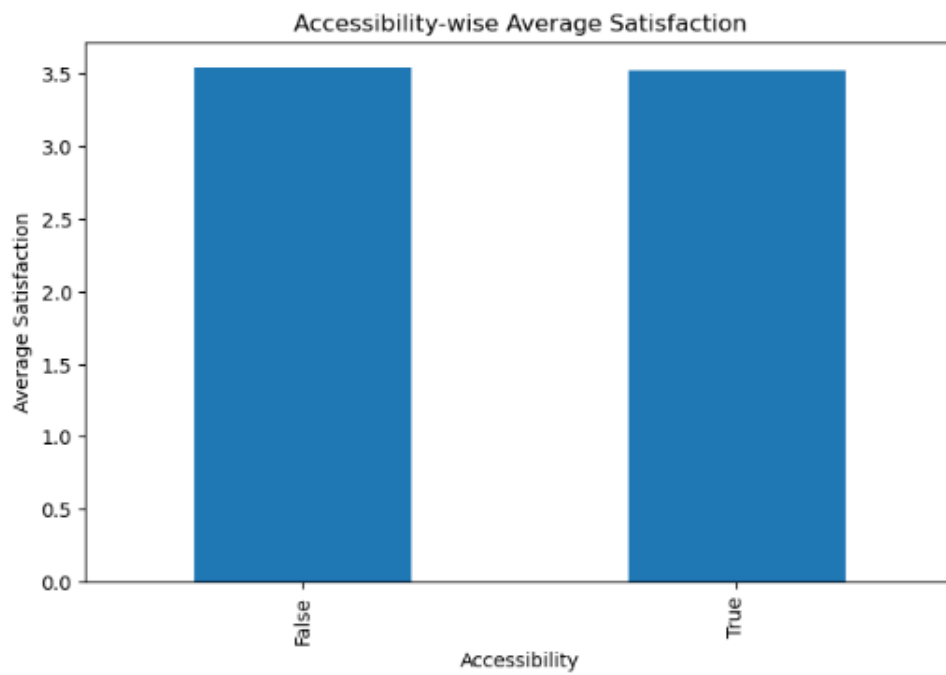
plt.xlabel("Accessibility")

plt.ylabel("Average Satisfaction")

plt.title("Accessibility-wise Average Satisfaction")

plt.show()

OUTPUT



RESULT

Thus, data inspection, filtering, sorting, descriptive statistical analysis, grouping, aggregation, and correlation analysis were successfully performed using Pandas. The analytical results were visualized and interpreted successfully.